

Laboratorio 2

TouchPad, WiFi y Servidor Web

Angelo Gauthier

Pilar Martinez

Ignacio Negri

Objetivos

Para esta primera parte del laboratorio 2, el objetivo es comenzar a trabajar con la placa de expansión ESP-LyraP-TouchA v1.1, interiorizándonos en la lectura de botones capacitivos mediante el periférico TouchPad del ESP32-S2-Kaluga-1. Se busca comprender la conexión física entre la placa base y la de extensión, el funcionamiento interno del TouchPad, cómo detecta variaciones de capacitancia e identificar el valor de umbral para presionar un botón del TouchA.

Se analizará la lectura de botones mediante polling e interrupciones, para comprender sus ventajas y desventajas, así como los casos en los que es más favorable utilizar un método o el otro dentro de un sistema embebido.

Se integrará el led rgb previamente trabajado en el Laboratorio 1 con la placa ESP-LyraP-TouchA v1.1.

Primera parte

Botones capacitivos (Touchpad):

Para conectar la placa de expansión ESP-LyraP-TouchA con la ESP32-S2-Kaluga-1 se utiliza el cable FPC entre el conector “Touch FPC” de la placa base y el conector FPC del Touchpad. Lo que debe revisarse es la posición del “Touch Switch” de la placa base.

Como se observa en la Figura 1, las señales ESP_TOUCH1 a ESP_TOUCH14 provenientes del conector FPC pasan a través de los switches SW3 y SW4, que determinan si los pines GPIO se utilizan como entradas táctiles (TH_GPIOx) o como GPIO externos (EXT_GPIOx).

Si esta se encuentra en posición off, los GPIO1 a GPIO14 se conectan a los canales táctiles del TouchPad, lo que permite el correcto funcionamiento de la placa de expansión. Si se encuentra en posición on, los GPIO también pueden ser utilizados con otros fines, por lo que pueden provocar un funcionamiento incorrecto en el TouchPad.

Para el uso del led rgb, se debe verificar que el rgb jumper este colocado. Cosa que se evaluó en el laboratorio anterior.

Los problemas conocidos para el ESP-LyraP-TouchA no son específicos de una falla propia del TouchPad, sino debido a la compatibilidad con otras placas de expansión del kit. Por ejemplo, al utilizar el TouchA con la pantalla LCD32, se comparten los pines IO6 e IO11, por lo que estos no deberían inicializarse. Al usarlo con la cámara, existen conflictos con IO1, IO2 e IO3. Si se usa con la placa de audio (8311A), también puede haber un problema con IO6 si se utiliza el pin BT_ADC de la placa de audio.

Por último, se comparten los pines IO39, IO40, IO41 e IO42 con la interfaz JTAG, provocando posibles fallas en el funcionamiento del JTAG al hacer uso del TouchPad.

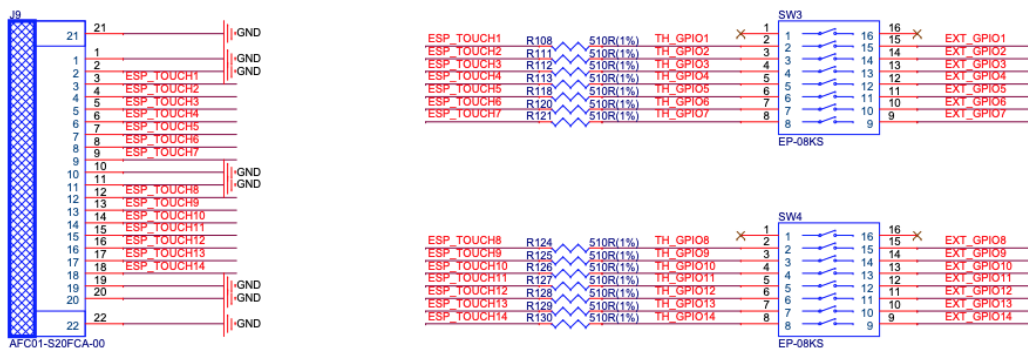


Figura 1: Imagen del conector FPC

Lectura de botones por polling:

Se diseñó en pseudocódigo un algoritmo que inicialice el TouchPad, lea periódicamente el estado de los 6 botones y detecte los eventos de presionado y liberado:

```

Inicializar TouchPad

Para cada botón:
    estado_anterior[i] ← LIBERADO

Mientras el sistema esté funcionando:
    Para cada botón i entre 0 y 5:
        estado_actual ← leer_boton(i)
        Si estado_actual = PRESIONADO
            y estado_anterior[i] = LIBERADO entonces
                detectar evento PRESIONADO
        Fin Si
        Si estado_actual = LIBERADO
            y estado_anterior[i] = PRESIONADO entonces
                detectar evento LIBERADO
        Fin Si
        estado_anterior[i] ← estado_actual
    Esperar un pequeño intervalo de tiempo
Fin Mientras
  
```

El periférico TouchPad del ESP32-S2 permite leer botones capacitivos sin necesidad de utilizar botones mecánicos. Cada botón se conecta a un canal del ESP32-S2. Su funcionamiento se basa en los cambios de capacitancia en los canales.

Cuando no existe un contacto, el TouchPad tiene un cierto de valor de capacitancia, formada por el propio botón capacitivo metálico, el circuito y la humedad del ambiente. Al apoyar un dedo, el cuerpo agrega capacitancia. Al aumentar, el proceso de carga y descarga se

realiza de manera más lenta, por lo tanto, cambia el valor que mide el periférico. El canal cuenta la cantidad de ciclos de reloj que dura una cantidad determinada de ciclos de carga y descarga.

Luego, el código compara ese valor con un límite (threshold) para decidir si el botón está presionado o no. Si el valor sobrepasa el threshold, el botón fue presionado. Para calibrarlo, primero se deben realizar medidas del TouchPad sin tocar los botones, para conocer un valor base, y después se presionan los botones y se mide nuevamente. Más tarde, se elige un valor intermedio (punto medio) entre ambas mediciones (base y presionado) para actuar de valor umbral (threshold).

En la detección por polling el código revisa constantemente el estado de los botones. Se fija si estos están presionados, si siguen presionados o si ya no lo están. La desventaja que presenta es que el ESP32-S2 tiene que estar revisando seguido, aunque no se presione ningún botón. También ocurre que, si el tiempo entre las revisiones es muy largo, puede perder pulsaciones breves.

Por otro lado, en la detección por interrupción, el código no pregunta periódicamente. Es el periférico quien avisa automáticamente cuando ocurre un evento, por ejemplo, cuando un botón supera el threshold. Allí se interrumpe el flujo del código y se ejecuta una determinada rutina de interrupción.

El método de polling conviene usarse en sistemas donde no existan muchas tareas al mismo tiempo, no se necesite una respuesta inmediata y el consumo excesivo de CPU no sea un problema.

El método de interrupción conviene usarse cuando se necesita una respuesta inmediata, cuando el ESP32-S2 está ocupado con otras tareas o si se quiere ahorrar en consumo del CPU. Además, sirve para que el sistema permanezca esperando y actúe solo cuando se pulsa un botón.

Finalmente, el polling es más simple pero menos eficiente, mientras que la interrupción es más eficiente y rápida, pero más compleja de programar.

Conclusión de la primera parte

Se logró comprender el funcionamiento de los botones capacitivos de la placa ESP-LyraP-TouchA y su integración con la placa ESP32-S2-Kaluga-1. Se identificó que la conexión física se realiza mediante un cable FPC y que, previamente, se debe posicionar al Touch Switch en off, para que los GPIO queden conectados a los canales del TouchPad.

También se concluye que los problemas de la placa no se deben a una falla propia de este, sino a que comparte pines con las otras placas de extensión del kit (pantalla, cámara, audio).

En cuanto al funcionamiento del TouchPad, se concluye que no detecta botones presionados de manera mecánica, sino que mide cambios en la capacitancia de cada canal. Al apoyar un dedo sobre un botón, la capacitancia aumenta y cambia el valor de ciclos de reloj

que dura una cantidad determinada de ciclos de carga y descarga. El periférico mide esa lectura y lo compara con el threshold para decidir si el botón está apretado o no.

Finalmente, se comparó el uso de detección por polling y detección por interrupción. Se concluye que el polling resulta más fácil de implementar y es adecuado para este laboratorio, ya que revisa constantemente el estado de los botones y asocia cada pulsación a un color visible en el led rgb. Sin embargo, es menos eficiente debido al exceso de uso del CPU. En cambio, las interrupciones, son más rápidas y eficientes, aunque es más complejo programarlas.

Por ende, la elección entre los dos métodos debe hacerse tomando en cuenta qué tan complejo es el programa, el consumo de CPU y la velocidad de respuesta que se requiere.

Segunda parte

Objetivos

Para esta segunda parte, el objetivo es utilizar la conectividad WiFi del ESP32-S2, con sus dos modos principales de operación: Access Point y Station. Para ello, se buscará información sobre la infraestructura de red de ESP-IDF y cómo iniciar la configuración WiFi.

Se investigará cómo la placa maneja las interfaces de red, los eventos WiFi y la asignación de direcciones IP. Se evaluará la diferencia entre el modo AP y el modo STA.

A su vez, otro objetivo será el de probar distintas configuraciones para la red, tales como: red abierta, red con contraseña WPA2, un SSID oculto y establecer un máximo de conexiones simultáneas.

A partir de estas pruebas, se observará qué ocurre cuando un dispositivo se conecta, cómo se asignan las direcciones IP mediante diferentes mecanismos y qué información se muestra en el monitor.

Infraestructura de red y event loop:

La capa de abstracción de red inicializada de ESP-IDF se llama ESP-NETIF. Funciona como intermediaria entre el driver de red y el stack TCP/IP, que es la parte del sistema que se encarga de que el ESP32-S2 pueda comunicarse por red.

ESP-NETIF es una capa de abstracción sobre el stack TCP/IP que permite manejar interfaces de red, direcciones IP, DHCP, DNS y eventos de red de una forma ordenada. Entonces, internamente, deja lista la infraestructura necesaria que se utilizará para poner, por ejemplo, una interfaz WiFi en modo estación (STA) o modo punto de acceso (AP).

Debe llamarse a `esp_netif_init()` antes de realizar cualquier operación de red, debido a que el sistema, de lo contrario, no tendría una estructura de red lista para manejar IP, DHCP, eventos de conexión o tráfico de paquetes.

Un event loop de ESP-IDF es un mecanismo para manejar eventos del sistema. Por ejemplo, cuando se inicia el WiFi, se obtiene una dirección IP o un dispositivo se conecta al AP, cada uno de estos eventos realiza un aviso.

La función `esp_event_loop_create_default()` crea el loop de eventos por defecto, que se utiliza por componentes WiFi e IP. Luego de dar aviso de los eventos, la aplicación puede utilizar handlers para llevar a cabo ciertas respuestas. Por ejemplo, cuando se obtiene una IP, se puede iniciar un servidor web.

Con respecto a los eventos `WIFI_EVENT_STA_CONNECTED`, `IP_EVENT_STA_GOT_IP`, `WIFI_EVENT_AP_STA_CONNECTED`:

- `WIFI_EVENT_STA_CONNECTED` permite saber si la conexión WiFi fue exitosa (Access Pointe externo).

- IP_EVENT_STA_GOT_IP indica que el ESP32-S2 obtuvo una dirección IP y que la red ya está lista para usar HTTP u otras formas de comunicación de red.
- WIFI_EVENT_AP_STACONNECTED permite registrar que un celular o PC entró a la red (AP) del ESP32-S2.

En el caso del modo STA no alcanza únicamente con WIFI_EVENT_STA_CONNECTED. Aunque el ESP32-S2 ya se haya conectado al Access Point, aún puede faltarle la dirección IP. Para levantar un servidor o utilizar HTTP se debe esperar a IP_EVENT_STA_GOT_IP.

Modo Access Point (AP):

Se diseñó en pseudocódigo la secuencia de inicialización completa de un AP: inicialización de netif, event loop, driver WiFi, configuración y arranque:

```

Inicializar NVS

Inicializar la capa de red:

    llamar a esp_netif_init()

Crear el event loop por defecto:

    llamar a esp_event_loop_create_default()

Crear la interfaz de red para modo AP:

    llamar a esp_netif_create_default_wifi_ap()

Inicializar el driver WiFi:

    crear configuración por defecto WIFI_INIT_CONFIG_DEFAULT()

    llamar a esp_wifi_init()

Registrar handlers de eventos WiFi:

    registrar evento WIFI_EVENT_AP_STACONNECTED

    registrar evento WIFI_EVENT_AP_STADISCONNECTED

Configurar parámetros del Access Point:

    definir SSID

    definir contraseña

    definir canal

    definir máximo de conexiones

    definir modo de autenticación

Aplicar configuración WiFi:

    llamar a esp_wifi_set_mode(WIFI_MODE_AP)

    llamar a esp_wifi_set_config(WIFI_IF_AP, configuración_ap)

Arrancar WiFi:

    llamar a esp_wifi_start()

Mostrar en monitor:

    SSID creado

    contraseña

    IP del AP

    cantidad máxima de conexiones permitidas

```

Campo	Tipo	Descripción
SSID	char[32]	Nombre de la red (se muestra en el escaneo de redes).
Password	char[64]	Contraseña de la red (mínimo 8 caracteres en WPA2).
Canal (channel)	uint8_t	Canal RF (1–13). En AP, determina el canal broadcast.
Seguridad	enum	OPEN, WPA_PSK, WPA2_PSK, WPA_WPA2_PSK, etc.
Max conexiones	uint8_t	Solo en AP: máximo de estaciones simultáneas (1–15).
SSID oculto	bool	Solo en AP: si es true el SSID no se anuncia en beacon frames.
Modo de autenticación	enum	WIFI_AUTH_OPEN, WIFI_AUTH_WPA2_PSK, entre otros.

Tabla 1: parámetros de la configuración de la API WIFI

Cuando un dispositivo se conecta al Access Point generado por el ESP32-S2, el monitor serie registra eventos WiFi relacionados con la conexión de estaciones. Entre ellos, puede aparecer el evento `WIFI_EVENT_AP_STACONNECTED`, indicando que un dispositivo externo se conectó exitosamente a la red creada. También puede mostrarse la dirección MAC del dispositivo conectado y la cantidad de estaciones actualmente asociadas al AP.

Además, el dispositivo que se conecta obtiene automáticamente una dirección IP. Por defecto el ESP32-S2 utiliza 192.168.1 como gateway del AP y actúa también como servidor DHCP. Por este motivo, asigna direcciones IP dentro de la red 192.168.4.x a los dispositivos conectados.

El mecanismo utilizado para la asignación de direcciones IP es DHCP como se mencionó anteriormente (Dynamic Host Configuration Protocol). Este permite entregar automáticamente una IP, máscara de subred y gateway sin necesidad de configuración manual.

En la Tabla 2, se evaluaron distintas configuraciones del modo estación Access Point, de manera de analizar el comportamiento del ESP32-S2 frente a diferentes parámetros de seguridad, visibilidad de red y cantidad máxima de conexiones simultáneas.

Configuración	¿Funciona?	Observaciones
Red abierta (sin contraseña)	si	La conexión es inmediata, el acceso queda restringido únicamente por el parámetro de máximo de conexiones simultáneas configurado en el AP
WPA2 con contraseña	si	Se requiere autenticación, aumenta la seguridad del Access Point
SSID oculto	si	No aparece automáticamente en el escaneo WiFi, debe ingresarse manualmente con el nombre de la red
Máximo 1 conexión simultánea	si	Solo un dispositivo puede conectarse al mismo tiempo, los demás son rechazados

Tabla 2: configuraciones de modo AP evaluadas en la practica
--

Modo estación (STA):

Se dise;o en pseudocódigo la secuencia de inicialización de una STA: qué cambia respecto del AP y qué eventos hay que manejar para saber cuándo se obtuvo dirección IP:

```

Inicializar NVS

Inicializar la capa de red:
    llamar a esp_netif_init()

Crear el event loop por defecto:
    llamar a esp_event_loop_create_default()

Crear la interfaz de red para modo estación:
    llamar a esp_netif_create_default_wifi_sta()

Inicializar el driver WiFi:
    crear configuración por defecto WIFI_INIT_CONFIG_DEFAULT()
    llamar a esp_wifi_init()

Registrar handlers de eventos:
    registrar WIFI_EVENT_STA_START
    registrar WIFI_EVENT_STA_CONNECTED
    registrar WIFI_EVENT_STA_DISCONNECTED
    registrar IP_EVENT_STA_GOT_IP

Configurar parámetros de la red:
    definir SSID de la red externa
    definir contraseña de la red externa

Aplicar configuración WiFi:
    llamar a esp_wifi_set_mode(WIFI_MODE_STA)
    llamar a esp_wifi_set_config(WIFI_IF_STA, configuración_sta)

Arrancar WiFi:
    llamar a esp_wifi_start()

Al recibir WIFI_EVENT_STA_START:
    intentar conectarse al Access Point externo
  
```

Al recibir WIFI_EVENT_STA_CONNECTED:

indicar que la conexión WiFi fue establecida

Al recibir IP_EVENT_STA_GOT_IP:

guardar o mostrar la dirección IP obtenida

considerar que la red ya está lista para usar HTTP u otros servicios

Al recibir WIFI_EVENT_STA_DISCONNECTED:

informar desconexión

intentar reconectar si corresponde

En modo estación (STA), el ESP32-S2 se conecta a una red WiFi ya existente actuando como un cliente más dentro de la red. A diferencia del modo AP, en este caso, el microcontrolador no genera una red propia ni asigna direcciones IP.

Cuando la conexión es exitosa, el monitor serie muestra eventos como WIFI_EVENT_STA_CONNECTED, indicando que el ESP32-S2 logró conectarse al AP externo. Luego aparece el evento IP_EVENT_STA_GOT_IP, señalando que el dispositivo obtuvo una dirección IP válida.

La dirección IP es asignada automáticamente por el router de la red mediante DHCP. Dependiendo de la configuración de la red, el ESP32-S2 puede obtener una IP como 192.168.1.x o 192.168.0.x.

Con respecto a la inicialización, tanto AP como STA utilizan esp_netif_init(), esp_event_loop_create_default() y la inicialización del driver WiFi. Sin embargo, en modo AP el ESP32-S2 crea su propia red y funciona como servidor DHCP, mientras que en modo STA debe conectarse a un AP externo y esperar a recibir una dirección IP antes de poder utilizar servicios de red.

En la Tabla 3, se evaluaron dos configuraciones en modo STA (red abierta y red con contraseña), de manera de comparar el comportamiento del ESP32-S2 frente a distintos niveles de seguridad de red.

Configuración	¿Funciona?	Observaciones
Red abierta (sin contraseña)	si	La conexión es rápida porque no requiere autenticación
WPA2 con contraseña	si	Requiere ingresar correctamente el SSID y contraseña para poder conectarse. Es más segura.

Tabla 3: configuraciones de modo STA evaluadas en la práctica

Implementación de la librería wifi_manager

Para organizar mejor el laboratorio y desacoplar la lógica de conectividad WiFi de la carpeta main, se implementó una librería propia denominada wifi_manager. Esta librería

contiene toda la configuración relacionada con la infraestructura de red, inicialización del WiFi y manejo de eventos del sistema.

Dentro de esta librería se concentró la lógica correspondiente a:

- Inicialización de `esp_netif`
- Creación del event loop mediante `esp_event_loop_create_default()`
- Configuración del driver WiFi
- Inicialización de los modos AP y STA
- Manejo de eventos WiFi e IP mediante handlers

De esta manera, el `main.c` quedará encargado únicamente de coordinar el comportamiento general del sistema, mientras que todos los detalles específicos de conectividad permanecen encapsulados dentro de la librería.

La utilización de `wifi_manager` permitió reutilizar código entre distintas partes del laboratorio. Eso simplificará la integración del servidor web con las partes realizadas anteriormente. Además, cualquier modificación relacionada con WiFi puede realizarse centralizadamente dentro del mismo módulo. Por ejemplo, se agregó que el usuario en `main.c` pueda elegir el modo de conexión, siendo este AP o STA.

Conclusión de la segunda parte

En esta segunda parte del laboratorio se logró implementar y comprender los dos principales modos de funcionamiento WiFi del ESP32-S2: AP y STA. Se analizó el funcionamiento de la infraestructura de red de ESP-IDF mediante `esp_netif` y el manejo de eventos utilizando event loop.

En modo AP, el microcontrolador fue capaz de generar su propia red WiFi y asignar direcciones IP automáticamente mediante DHCP a los dispositivos conectados. También se evaluaron distintas configuraciones de seguridad y visibilidad de la red, verificando su comportamiento práctico.

Por otro lado, en modo STA el ESP32-S2 logró conectarse correctamente a una red WiFi existente y obtener una dirección IP asignada por el router. Se observó la importancia de eventos como el `IP_EVENT_STA_GOT_IP` para confirmar que la red ya se encuentra disponible para utilizar servicios HTTP u otras comunicaciones.

Finalmente, se concluye que el modo AP resulta útil cuando se desea que el microcontrolador funcione de manera independiente creando su propia red, mientras que el modo STA permite integrar la placa dentro de una infraestructura WiFi ya existente.

Para concluir, el uso de la arquitectura modular termina favoreciendo la integración progresiva del proyecto. Ya que, más adelante, permitió incorporar el servidor HTTP y la comunicación con la interfaz web sin modificar la estructura principal de la aplicación.

Tercera parte

Objetivos

El objetivo que se plantea es el de implementar un servidor web embebido de tipo HTTP utilizando la API `esp_http_server`. A partir de la parte anterior, se hará uso de la conectividad WiFi del ESP32-S2 para que, desde una interfaz web, tanto desde PC como de un celular, se pueda acceder a la placa y permita controlar el led rgb de forma remota.

Además, se integrará el funcionamiento del TouchPad al del led rgb y la interfaz web en un solo sistema. El objetivo será poder modificar el led desde la página web y desde el TouchPad.

Por otro lado, también se responderán ciertas preguntas como: ¿Desde qué dirección IP se accedió al servidor?; ¿Qué es cJSON y para qué sirve?; ¿Qué mecanismo se podría utilizar para que la página web se actualice sola cuando el led cambie?

Implementación de la librería `web_server`

De forma similar a la conectividad WiFi, la lógica correspondiente al servidor HTTP se encapsulo dentro de una librería independiente denominada `web_server`. El objetivo fue desacoplar la implementacion del servidor web del `main.c` y mantener la misma arquitectura modular que es más fácil de modificar y mantener.

La librería `web_server` quedo encargada de:

- Inicializar el servidor HTTP usando la API `esp_http_server`
- Registrar los distintos URI handlers
- Servir los archivos web embebidos (`index.html`, `style.css` y `app.js`)
- Manejar las solicitudes GET y POST
- Administrar la comunicación entre la interfaz web y el ESP32-S2

Dentro de este módulo se implementaron endpoints como GET `/led` (utilizado para consultar el estado actual del led) y POST `/led` (utilizado para recibir comandos desde la página web en formato JSON y modificar el color del LED).

Además, la librería se integró con cJSON para parsear los mensajes recibidos desde el navegador y convertirlos en estructuras de datos utilizables por el programa C. Esta implementacion ayudo a mantener separadas las responsabilidades de red, interfaz y control del hardware.

“Hola mundo” web

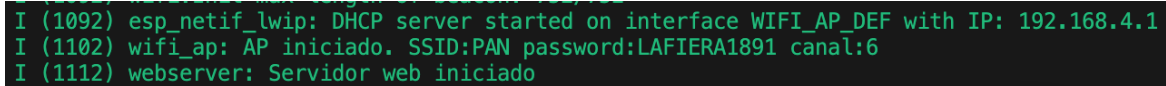
Utilizando la librería descrita anteriormente, se configuro un servidor HTTP básico capaz de responder solicitudes desde un navegador web. Para acceder al servidor web embebido se utilizó la dirección IP 192.168.4.1, correspondiente al ESP32-S2 funcionando en modo Access Point.

Cuando la placa opera en modo AP, crea su propia red WiFi ya actúa simultáneamente como gateway y servidor DHCP. Por este motivo, el propio ESP32-S2 utiliza por defecto la dirección

IP detallada previamente, mientras que los dispositivos que se conectan a la red reciben automáticamente direcciones IP dentro del rango 192.168.4.x.

Una vez conectado un celular o PC a la red creada por el ESP32-S2 (WIFI_SSID: "PAN"; WIFI_PASS: "LAFIERA1891"), fue posible acceder desde el navegador ingresando la dirección: <http://192.168.4.1>.

De esta manera, la página “Hola Mundo” pudo visualizarse correctamente tanto desde nuestros celulares como desde una PC conectado al AP anterior tal cual lo muestra la Figura 2.



```
I (1092) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (1102) wifi_ap: AP iniciado. SSID: PAN password: LAFIERA1891 canal: 6
I (1112) webserver: Servidor web iniciado
```

Figura 2: imagen del monitor mostrando la conexión exitosamente establecida

Página web embebida

A partir de la infraestructura del servidor HTTP ya implementada mediante la librería `web_server`, se incorporó soporte para servir archivos web embebidos directamente desde el binario de la aplicación.

En lugar de generar el código HTML directamente como un string dentro del programa en C, se utilizó la funcionalidad de `EMBED_FILES` de ESP-IDF para incorporar archivos web directamente al binario de la aplicación.

Para ello, se creó una carpeta `web/` dentro de `main/`, la cual contiene los archivos `index.html`, `style.css` y `app.js`. Luego, dichos archivos fueron agregados al `CMakeLists.txt` mediante la directiva `EMBED_FILES`, permitiendo que el ESP32-S2 los sirva desde el servidor HTTP embebido.

El archivo `index.html` contiene la estructura principal de la página, `style.css` define los estilos visuales y `app.js` implementa la lógica de interacción entre la interfaz web y el ESP32-S2 mediante solicitudes HTTP.

Control del LED RGB desde el navegador

Sobre los endpoints HTTP implementados previamente en la librería `web_server`, se agregó soporte para intercambio de información utilizando la librería `cJSON` en C, la cual permite trabajar con datos en formato JSON. Este es un formato utilizado para el intercambio de información entre un cliente y un servidor en aplicaciones web y sistemas embebidos conectados a red.

En este laboratorio, el ESP32-S2 recibe desde la página web mensajes HTTP que contienen información del color del LED RGB en formato JSON. Debido a que, en C, los datos llegan como un string de texto, es necesario parsearlo. Eso significa interpretarlo y extraer de allí los valores de cada campo.

El uso de `cJSON`, permite realizar este proceso de forma más ordenada y segura que haciéndolo manualmente con strings. La librería convierte el texto JSON en estructuras de

datos que permiten acceder directamente a campos específicos como, por ejemplo, los valores r, g y b correspondientes al color del LED.

El analizar manualmente los strings puede llevar a errores o mensajes incompletos. Además, el código resultaría más difícil de mantener y menos escalable.

Como alternativas para serialización de datos en sistemas embebidos, pueden utilizarse formatos simples como texto plano, CSV o pares clave-valor. También existen formatos binarios, como Protocol Buffers o MessagePack, que ocupan menos memoria y ancho de banda, aunque son más difíciles no solo de leer para una persona, sino también de implementar.

Integración: Touchpad + LED + web

Para integrar el código del TouchPad desarrollado en la Primera Parte, se incorporó la librería touch dentro de la carpeta components/, junto con las librerías ya existentes de rgb_led, delay, led_strip, web_server y wifi_manager.

La librería touch quedó encargada de inicializar el periférico TouchPad del ESP32-S2 y de ofrecer una función simple para consultar si un botón está presionado. De esta manera, el main.c no necesita manejar directamente los detalles del driver TouchPad, sino que solamente llama a funciones como: “touchpad_init”, “touchpad_is_pressed” y “touchpad_update_led”.

Luego, en el main.c, se inicializó el TouchPad junto con el LED RGB, el WiFi y el servidor web. Dentro del ciclo principal del programa se consultó periódicamente el estado de los botones mediante polling. Cuando se detecta que un botón fue presionado, se ejecuta una acción asociada sobre el LED RGB, por ejemplo, cambiar su color o apagarlo.

Además, para que el sistema sea consistente con la interfaz web, cada vez que el LED cambia por apretar un botón del TouchPad, también se actualiza la variable global que guarda el estado actual del LED. De esta forma, cuando la página web consulta el endpoint GET /led, recibe el último estado real del LED, sin importar si fue modificado desde la web o desde los botones físicos.

Así, no solo el TouchPad quedó integrado como una segunda forma de controlar el LED RGB, funcionando en conjunto con la interfaz web, sino que también de alguna manera, la página web se actualiza cuando el LED cambia por un botón.

Si bien el mecanismo utilizado termina siendo más simple de implementar, genera consultas constantes al servidor, incluso cuando no existen cambios en el estado del sistema. Para ello, hay ideas más eficientes para aplicaciones en tiempo real. Una posible solución para que la página web se actualice automáticamente cuando el LED cambia por un botón es utilizar WebSockets o Server-Sent Events (SSE).

La idea consiste en mantener una conexión permanente entre el navegador y el ESP32-S2. De esta manera, cuando el estado del LED cambie debido a una pulsación en el TouchPad, el ESP32-S2 podría enviar automáticamente el nuevo estado a la página web sin necesidad de que el usuario recargue la página manualmente. Esto permitiría mantener sincronizados en

tiempo real el TouchPad, el LED RGB y la interfaz web. La página web final se muestra en la Figura 3.



Figura 3: página web que maneja el LED RGB del ESP32-S2

Conclusión de la tercera parte

En esta tercera parte, se logró implementar un servidor web embebido utilizando la API `esp_http_server` de ESP-IDF. A partir de la conectividad WiFi del ESP32-S2, fue posible acceder desde un navegador web a una interfaz que permite visualizar y controlar el estado del LED RGB de manera remota, tanto desde una computadora como desde un celular conectado a la red de manera AP o STA.

Se integraron tecnologías web como HTML, CSS y JavaScript con endpoints HTTP implementados en C dentro del ESP32-S2. Mediante solicitudes GET y POST, la página web pudo consultar y modificar el estado del LED RGB utilizando mensajes en formato JSON. Para implementar dichos mensajes se utilizó la librería `cJSON`, permitiendo un manejo más ordenado, seguro y escalable de la información intercambiada entre el cliente y el servidor.

Además, se integró el código del TouchPad desarrollado al comienzo del laboratorio, incorporando la librería `touch` como un componente independiente del `main.c`. De esta forma, el sistema pudo controlar el LED RGB tanto desde los botones capacitivos como desde la interfaz web, manteniendo actualizado el estado del sistema independientemente del origen del cambio.

La sincronización de la página web con el estado del LED se realizó mediante polling al endpoint `GET / led`, permitiendo que la interfaz consulte periódicamente el estado actual del sistema. Si bien este mecanismo resultó suficiente para este laboratorio, se evaluaron ideas como WebSockets y SSE que podrían mejorar la eficiencia y permitir actualizaciones efectivas en tiempo real.

Finalmente, la modularización del proyecto mediante librerías independientes para wifi, TouchPad, LED RGB y el servidor web, permitió desarrollar una arquitectura más organizada, reutilizable y escalable. Esto facilitó la integración progresiva de funcionalidades a lo largo de todo el laboratorio.

Bibliografía:

- *ESP-IDF Programming Guide - ESP32-S2* - — *ESP-IDF Programming Guide v4.3 documentation.* (s. f.). <https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32s2/index.html>
- *ESP32-S2-Kaluga-1 Kit v1.2 - ESP32-S2* - — *ESP-IDF Programming Guide v4.3 documentation.* (s. f.). <https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32s2/hw-reference/esp32s2/user-guide-esp32-s2-kaluga-1-kit-v1.2.html>
- Hoja de datos: sch_esp32-s2-kaluga-1_v1_2_2020
- *W3Schools.com.* (s. f.). <https://www.w3schools.com/>